

Programación Orientada a Objetos

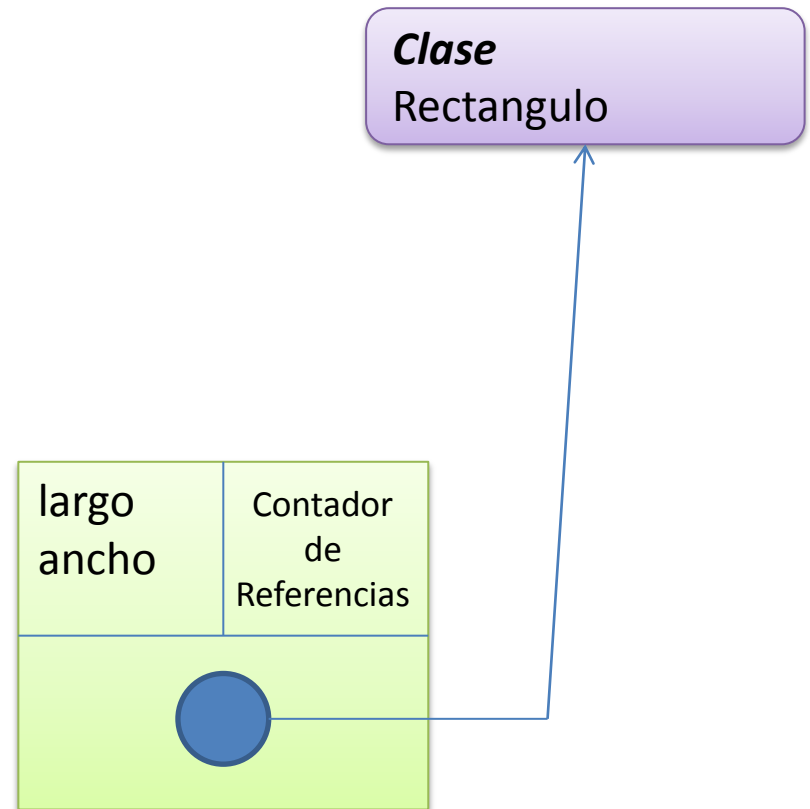
Primeros conceptos de OOP
tkinter

Definición

- *La programación orientada a objetos o POO (OOP según sus siglas en inglés) es un paradigma de programación que usa objetos y sus interacciones, para diseñar aplicaciones y programas informáticos. (wikipedia).*
- *Los objetos implican:*
 - *Un estado*
 - *Un comportamiento*
 - *Una identidad*

Características de los objetos

- El estado de un objeto corresponde a un conjunto de propiedades o atributos.
- En Python todo objeto está compuesto por sus atributos, un puntero o referencia a la definición del objeto –Clase– (pueden ser varias) y un contador que indica el número de veces que el objeto es referenciado.
- Recuerde que el contador de referencias es utilizado por el recolector de basura.



Instancias

- Los objetos pertenecientes a una clase reciben el nombre de instancias.
- Suponga que *Rectangulo* es una clase.
 - `a = Rectangulo(10, 20)` crea una instancia de la clase *Rectangulo* y la dirección de esa instancia se almacena en `a`.
- Observe que para crear una instancia de *Rectángulo* se requirieron dos valores numéricos.
 - En este caso utilizar `a = Rectangulo()` genera un error.

Haciendo uso de las propiedades de un objeto

- Suponga que *a* mantiene la dirección de una instancia de la clase rectángulo, y que en esta se definieron las propiedades largo y ancho.
 - a.largo y a.ancho refieren a las propiedades del objeto

Identidad

- La identidad de un objeto corresponde a un valor entero único que identifica al objeto.
- En C-Python la identidad del objeto corresponde a su dirección en memoria.
- `id(objeto)` retorna su identidad.
- Observe los siguientes ejemplos.

```
>>> id(1)
505416168
>>> a = 1
>>> id(a)
505416168
>>> b = 1
>>> id(b)
505416168

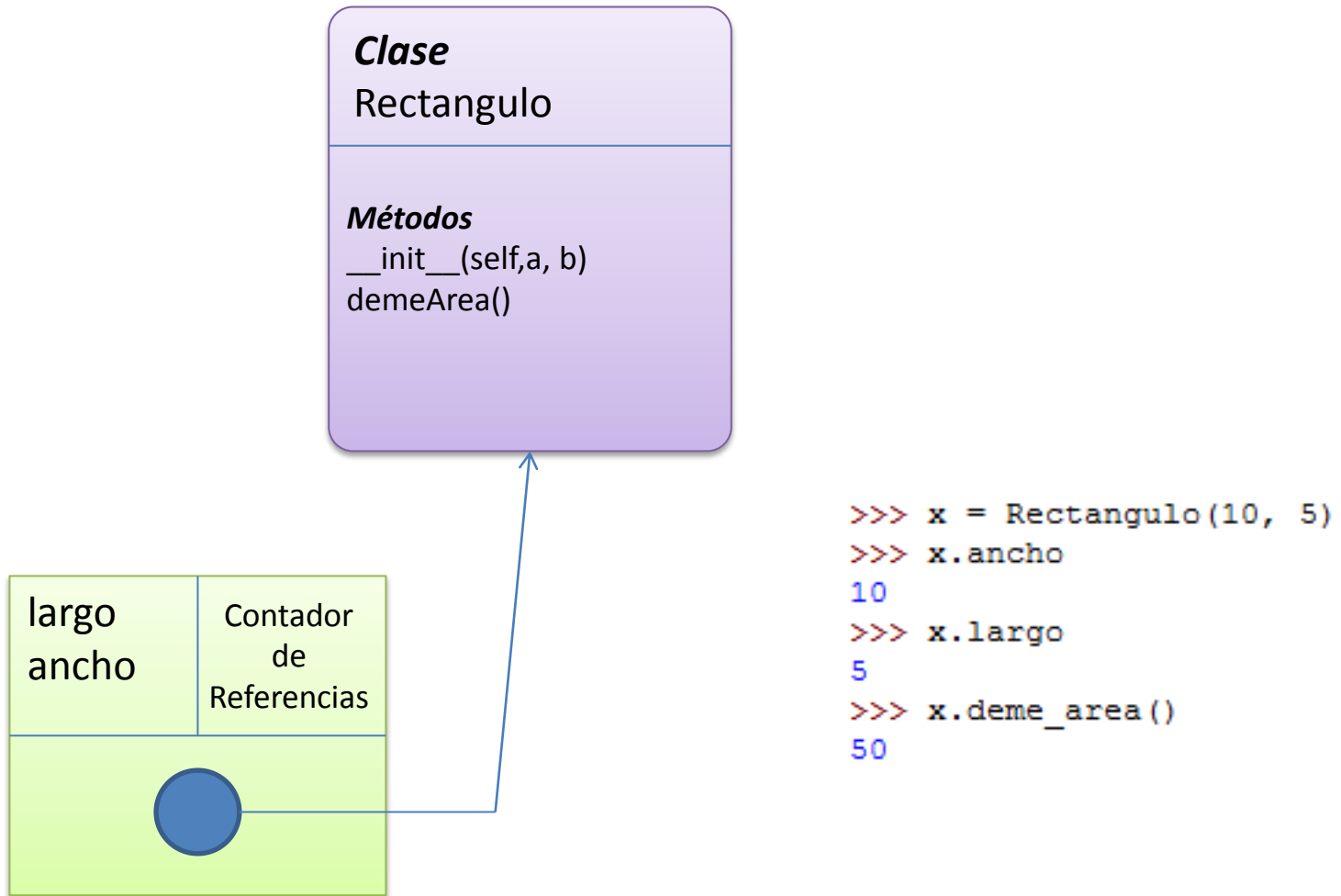
>>> a = "casa"
>>> b = "casa"
>>> a is b
True
>>> id(a)
35765312
>>> id(b)
35765312

>>> b = "esta es una tira muy larga"
>>> c = "esta es una tira muy larga"
>>> b is c
False
>>> id(b)
35908736
>>> id(c)
35908992
>>> b == c
True
```

Clases

- Una clase define las propiedades y el comportamiento de los objetos que serán sus instancias.
- El comportamiento corresponde a un conjunto de *métodos* (funciones) que permiten manipular los objetos.

La Clase Rectángulo



Definición de un clase

```
class Rectangulo():  
    def __init__(self, a, b):  
        self.ancho = a  
        self.largo = b  
    def deme_area(self):  
        return self.ancho * self.largo
```

Nombre de la clase.

El método `__init__` se invoca en forma automática cuando se crea una nueva instancia de la clase.
Se dice que `__init__` es un constructor.

Todos los métodos de la clase reciben en forma automática el parámetro `self`, el cual representa el objeto o instancia que está siendo manipulado.

Creación de instancias

```
class Rectangulo():  
    def __init__(self, a, b):  
        self.ancho = a  
        self.largo = b  
    def deme_area(self):  
        return self.ancho * self.largo
```

En este momento se crean los atributos.

```
>>> x = Rectangulo(2, 4)
```

Qué pasa si omitimos self en el `__init__`?

```
class Rectangulo():  
    def __init__(self, a, b):  
        self.anchos = a  
        self.largo = b  
    def deme_area(self):  
        return self.anchos * self.largo
```

vrs

```
class Rectangulo():  
    def __init__(self, a, b):  
        anchos = a  
        largo = b  
    def deme_area(self):  
        return self.anchos * self.largo
```

- (1) En la definición de la clase no se genera ningún error
- (2) Pero cuando utilizamos el método `deme_area` se produce un error.

Porqué?

En el segundo caso, en el método `__init__`, `anchos` y `largo` son tomadas como locales ...

Y que pasa si usamos:

```
class Rectangulo():  
    def __init__(self, a, b):  
        anchos = a  
        largo = b  
    def deme_area(self):  
        return anchos * largo
```

Igual, genera error ...

Una clase puede no tener `__init__`

El método `__init__` puede no estar presente en la definición de una clase.

A pesar de ello siempre se van a crear instancias de la clase.

Esto nos lleva a la siguiente verdad de las clases en Python...

La verdad sobre las propiedades o atributos de los objetos en Python

- En Python las variables se definen en el momento de asignarles un valor (realmente se crea el objeto y su dirección se almacena en la variable).
- Lo mismo sucede con los atributos de los objetos !!

```
class Repositorio():  
    pass
```

```
>>> x = Repositorio()  
>>> x.uno = 10  
>>> x.dos = 20  
>>> x.tres = lambda y : x.uno * x.dos + y  
>>> x.tres(4)  
204
```

```
>>> x = Repositorio()  
>>> x.uno = 10  
>>> y = Repositorio()  
>>> y.uno
```

```
Traceback (most recent call last):
```

```
File "<pyshell#58>", line 1, in <module>
```

```
y.uno
```

```
AttributeError: 'Repositorio' object has no attribute 'uno'
```

Herencia

- Una clase puede ser derivada de otra clase.
- La subclase heredará todos las propiedades y métodos de la superclase.

```
class Base:
    def __init__(self, a):
        self.nombre = a
    def deme_nombre(self):
        return self.nombre

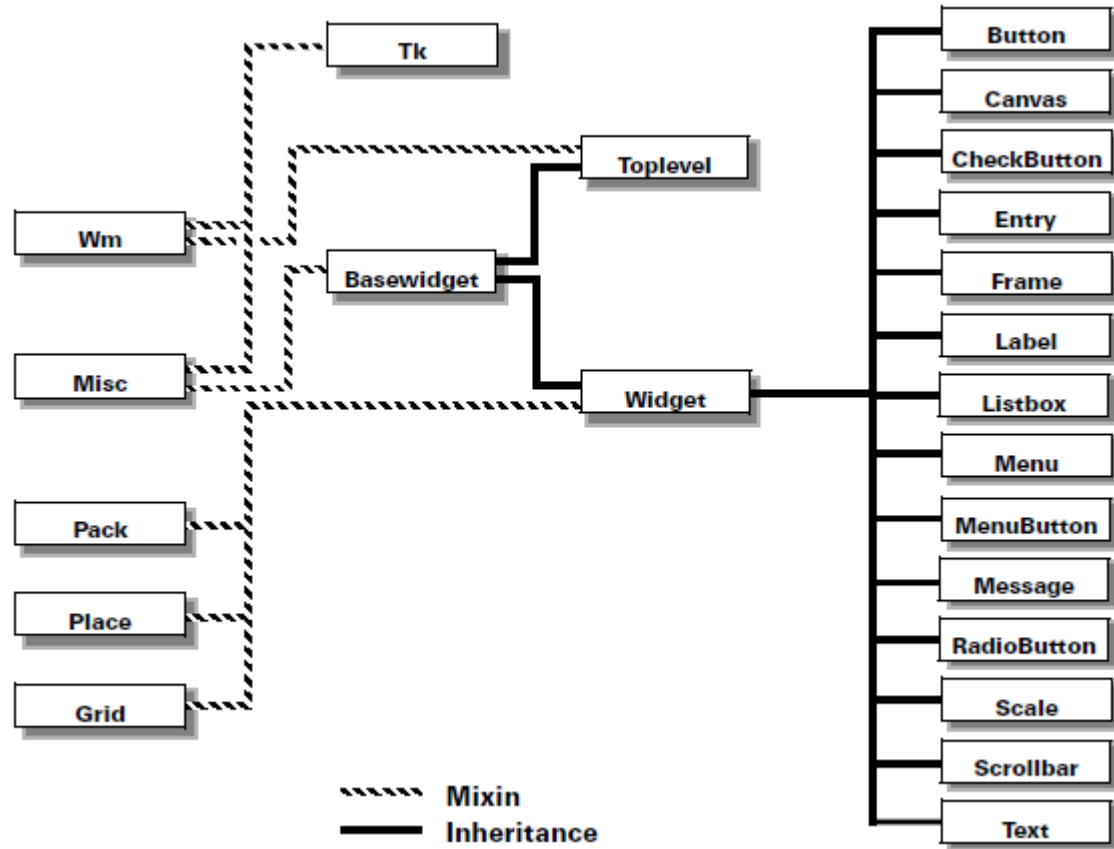
class Subclase(Base):
    def __init__(self, a, b):
        Base.__init__(self, a)
        self.apellido = b
    def deme_completo(self):
        return self.nombre + " " + self.apellido
```

```
>>> e = Subclase("Jose", "Perez")
>>> e.nombre
'Jose'
>>> e.apellido
'Perez'
>>> e.deme_completo()
'Jose Perez'
>>> f = Base("Jose")
>>> f.deme_nombre()
'Jose'
```

tkinter

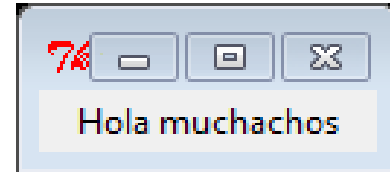
- tkinter provee a las aplicaciones Python de un interfaz gráfica.
- tkinter es la interfaz para Tk, un toolkit para Tcl/Tk. Tcl/Tk es una facilidad de scripting y de gráficos desarrollada por John Ousterhout quien formaba parte de la Universidad de Berkeley en California y que luego pasó a Sun Microsystems. Actualmente Tcl/Tk es desarrollado por Scriptics Corporation (de la cual Ousterhout fue fundador).

Jerarquía de tkinter



Nuestra primera aplicación con tkinter

```
from tkinter import *  
  
root = Tk()  
  
w = Label(root, text="Hola muchachos")  
w.pack()  
  
root.mainloop()
```



Segunda aplicación

```
from tkinter import *

class Application(Frame):
    def say_hi(self):
        print("Hola !")

    def createWidgets(self):
        self.QUIT = Button(self)
        self.QUIT["text"] = "Salir"
        self.QUIT["fg"] = "red"
        self.QUIT["command"] = self.quit

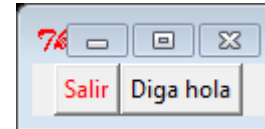
        self.QUIT.pack({"side": "left"})

        self.hi_there = Button(self)
        self.hi_there["text"] = "Diga hola"
        self.hi_there["command"] = self.say_hi

        self.hi_there.pack({"side": "left"})

    def __init__(self, master=None):
        Frame.__init__(self, master)
        self.pack()
        self.createWidgets()

root = Tk()
app = Application(master=root)
app.mainloop()
root.destroy()
```



```
>>>
Hola !
Hola !
>>> |
```